

Lecture 10: The Loader and LC-3 Assembly

Date: Thursday, June 18th, 2026

Reading: Patt & Patel 3rd ed.: Sec. 7.1-7.2

Announcements:

- HW4 due Saturday
- HW5 out Saturday or Sunday
- Quiz 2 on Wednesday
- Project 1 out today or tomorrow

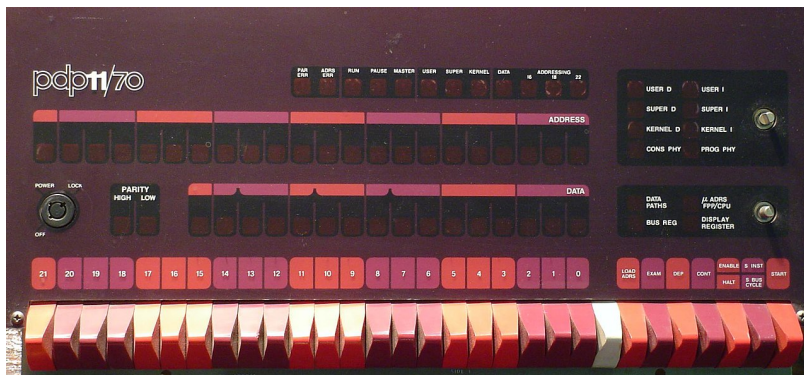
Part 1: Object Files and the Loader

Learning objectives for Lecture 10 Part 1:

- Course learning objective #5: **Assemble** a symbolic assembly language and describe its structure and purpose.
 - **Initialize** memory by reading an LC-3 object file.
 - **Write** LC-3 object files that describe how to initialize memory with specific instructions and data.
- Course learning objective #6: **Define** basic concepts of computer systems, such as the runtime stack and simple I/O devices.
 - **Explain** the function of the loader in the LC-3 operating system.

Initializing Memory by Hand

Thus far, we have assumed that user programs was magically placed in memory. But practically, someone (or something) must put programs into memory. On the PDP-11 you could load a program into memory yourself by flipping the switches on the front panel:



Let's try this procedure on the LC-3. As an example, let's initialize memory with the program from Lecture 7 that computes $R1 \leftarrow R2 - R3$:

Address	Assembly Syntax	Value at Address (Binary)	Value at Address (Hex)
0x3000	NOT R3, R3	1001 0110 1111 1111 ₂	96FF ₁₆
0x3001	ADD R3, R3, 1	0001 0110 1110 0001 ₂	16E1 ₁₆
0x3002	ADD R1, R2, R3	0001 0010 1000 0011 ₂	1283 ₁₆
0x3003	HALT	1111 0000 0010 0101 ₂	F025 ₁₆

For convenience, we will try it on **LC3Tools**, the LC-3 simulator used in this course. We can simply go to “Simulator” and click some memory locations to change their values (you may find it helpful to click “Reinitialize Machine” to set all memory/registers to 0):

LC3Tools

Registers

Reg	Hex	Decimal	ASCII / M
R0	x0000	0	
R1	x0000	0	
R2	x0000	0	
R3	x0000	0	
R4	x0000	0	
R5	x0000	0	
R6	x0000	0	
R7	x0000	0	
PSR	x8002	-32766	CC: Z
PC	x3000	12288	
MCR	x0000	0	

Memory

BP	PC	Address	Hex	Decimal	Label	Instructions
1	▶	x3000	x96FF	-26881		*NOT R3, R3
1	▶	x3001	x16E1	5857		*ADD R3, R3, #1
1	▶	x3002	x1283	4739		*ADD R1, R2, R3
1	▶	x3003				
1	▶	x3004				
1	▶	x3005				
1	▶	x3006				
1	▶	x3007	x0000	0		
1	▶	x3008	x0000	0		
1	▶	x3009	x0000	0		
1	▶	x300A	x0000	0		
1	▶	x300B	x0000	0		
1	▶	x300C	x0000	0		
1	▶	x300D	x0000	0		
1	▶	x300E	x0000	0		
1	▶	x300F	x0000	0		
1	▶	x3010	x0000	0		

Console

Debugger

Jump to Location

If we initialize R2 to 4 and R3 to 1 by clicking on their values on the register pane, then pressing the “Play” button yields the expected answer of $4 - 1 = 3$ in R1:

Registers

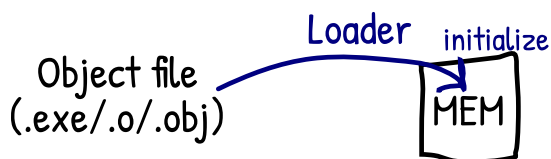
Reg:	Hex	Decimal	ASCII / Misc
R0	x0000	0	
R1	x0003	3	
R2	x0004	4	

Now, let’s discuss a more scalable way to initialize memory.

The Loader

As we have just seen, placing code in memory by initializing memory manually *can* work, provided the computer supports it as the PDP-11 did. However, there is a glaring problem with this approach: realistic programs (e.g., web browsers and compilers) span hundreds of thousands, if not millions, of memory locations. Manually initializing memory to run realistic programs is tedious and error-prone.

To address this, your operating system includes a program called a **loader** that initializes memory according to what an **object file** on your hard disk specifies, as shown in the drawing below. (Indeed, even on a PDP-11, the tiny program you placed in memory by hand was actually a loader intended to load in the *actual* program you wanted to run.)



The loader in the LC-3 operating system expects an object file that looks like this:

LC-3 OBJ FILE

.TEXT

```

3000
4
96FF
16E1
1283
F025

```

The line `.TEXT` starts a *section* named “*text*”. The name “text” is an awful historical name, but for our purposes right now, the “text” section contains the words with which the loader should initialize memory. The word `0x3000` indicates that **the window of memory to initialize** begins at address `0x3000`, and the word `0x0004` indicates that this window is four memory locations long, i.e., there are four memory locations to initialize. Then the next four words are the the desired values with which to initialize memory. (In this example, they are the same four instructions from above.) If we save this file as `diff.obj` and open it in LC3Tools, we see that the loader initializes memory as desired:

Memory

BP	PC	Address	Hex	Decimal	Label
1	▶	x3000	x96FF	-26881	
1	▶	x3001	x16E1	5857	
1	▶	x3002	x1283	4739	
1	▶	x3003	xF025	-4059	

In fact, we can even specify multiple windows of memory to initialize in the text section, like this:

LC-3 OBJ FILE

```

.TEXT
3000
4
96FF
16E1
1283
F025

3008
2
DEAD
BEEF

```

If we load this object file in LC3Tools, the the loader initializes both of those memory locations starting at `0x3008` too:

Memory

BP	PC	Address	Hex	Decimal	Label
1	▶	x3000	x96FF	-26881	
1	▶	x3001	x16E1	5857	
1	▶	x3002	x1283	4739	
1	▶	x3003	xF025	-4059	
1	▶	x3004	x4EE5	20197	
1	▶	x3005	x4901	18689	
1	▶	x3006	x52D7	21207	
1	▶	x3007	x161A	5658	
1	▶	x3008	xDEAD	-8531	
1	▶	x3009	xBEEF	-16657	
1	▶	x300A	x5BB3	23475	
1	▶	x300B	x142F	5167	

If you have ever seen an `.exe` file (an *executable*), know that it is simply an object file describing how memory should be initialized.

Writing an object file by hand as we have done here is highly unusual and only feasible because the LC-3 object file format is designed to be somewhat human-readable. But in the real world, in the same way that the encoding of a JPEG or PNG image is more complicated than a 2D array, a realistic object file format is also much, much more

complicated than the LC-3 object file format. Thus, we need a piece of software that can create object files for us without requiring detailed knowledge of the object file format. By analogy, we mean this in the same sense that an image editor (e.g., Photoshop) can encode images as JPEGs without the user having any knowledge of details of the JPEG encoding. Better yet, this piece of software could allow specifying how memory is initialized more conveniently than writing out e.g. encoded instructions as hexadecimal. We're in luck — this program exists, and it is called an *assembler*, discussed in the next part of this lecture.

Part 2: Assembly

Learning objectives for Lecture 10 Part 1:

- Course learning objective #3: **Analyze** the structure and examine the operation of a basic von Neumann CPU.
 - **Define** architectural and microarchitectural state.
 - **Classify** storage elements in the LC-3 datapath as either architectural or microarchitectural state.
- Course learning objective #5: **Assemble** a symbolic assembly language and describe its structure and purpose.
 - **Explain** the purpose of assembly language.
 - **Assemble** LC-3 assembly code, including assembly code with labels, to an LC-3 object file.
 - **Compile** pseudocode with variables, loops, and conditionals to assembly code.

Architectural versus Microarchitectural State

Let's review who we've acted as so far across the whole semester. When designing circuits with transistors, we acted as an electrical/computer engineer. In designing the LC-3 microarchitecture, we acted as a computer architect. Now, when we move to programming the LC-3, we leap to a higher level of abstraction, acting finally as *programmers*. The only catch is that we will program at a lower level of abstraction than usual; we will be directly manipulating architectural state as an operating system developer, security researcher, or '60s/'70s programmer would.

Architectural state is the portion of state of the CPU defined in the ISA specification. That is, architectural state is the abstraction of CPU state provided explicitly to the programmer. **Microarchitectural state** is the remainder of the state of the CPU, i.e., state that is a microarchitecture-specific implementation detail. Here are some examples from the LC-3 datapath:

Architectural State	Microarchitectural State
R0-R7	Current LC-3 FSM State
PC	MAR/MDR
CC	IR
Memory Locations	

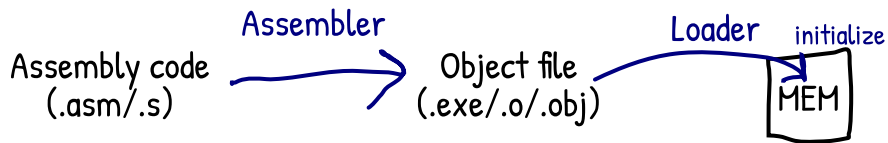
So when I said assembly programmers directly manipulate architectural state, I mean that programmers are reasoning in terms of values in general-purpose registers (temporaries, R0-R7) or the PC. This makes assembly programming live at a very low level of abstraction. Compare the abstractions provided by LC-3 assembly to those in language you've used in prerequisite courses:

LC-3 Assembly Abstractions	Python/Java Abstractions
R0-R7	Variables
Instructions	Statements/expressions
CC	Loops/conditionals
Memory locations	Functions/methods

Now, let's discuss assembly language in more detail.

Assembly Language

Assembly code describes how memory should be initialized. A piece of software called an *assembler* converts assembly to an object file as shown below. This process is called *assembling* the code.



As mentioned in the previous part of this lecture, there are two motivations for assembly language:

1. Writing object files byte-by-byte by hand is tedious and demands of the programmer unnecessary knowledge of the object file encoding. (By analogy, using Photoshop shouldn't require you know the JPEG encoding by heart.)
2. Writing instruction encodings by hand bit-by-bit is tedious. Instead, it is more convenient to write assembly syntax (e.g., `ADD R1, R2, R3` is more convenient to write than `0x1283`).

In assembly language, the subtraction code from the first part of lecture can be written as follows:

```
.ORIG x3000
NOT R3, R3
ADD R3, R3, 1
ADD R1, R2, R3
HALT
.END
```

If I save this code to `diff.asm` and load it in LC3Tools, I can press “Assemble” to run the assembler and generate the following object file, `diff.obj`:

LC-3 OBJ FILE

```
.TEXT
3000
4
96FF
16E1
1283
F025

.SYMBOL

.LINKER_INFO

.DEBUG
# DEBUG SYMBOLS FOR LC3TOOLS
```

```
=====
LINE | ADDR | SOURCE
  0 | ??? | .ORIG x3000\n
  1 | 3000 | NOT R3, R3\n
  2 | 3001 | ADD R3, R3, 1\n
  3 | 3002 | ADD R1, R2, R3\n
  4 | 3003 | HALT\n
  5 | ??? | .END\n
  6 | ??? | 
=====
```

Notice the `.TEXT` section is exactly the same as the one we wrote by hand in Part 1! We will discuss the new `.SYMBOL` and `.LINKER_INFO` sections later. Notice that *debug symbols* — mappings of addresses to lines of assembly, and vice

versa — are also present in the .DEBUG section. It is common for object files to contain debug symbols for debuggers such as the one in LC3Tools.

Assembler Directives

What are .ORIG and .END in the assembly code above? They are *assembler directives*. These are directions to the assembler for creating the object file; assembler directives are *NOT* LC-3 instructions.

- .orig x3000 / .end: Define a window of memory starting at address 0x3000.
- .fill 420: Encode 420_{10} as 16-bit two's complement and put the resulting word in memory (e.g., 0x01A4)
- .blkw 7: Reserve 7 uninitialized memory locations (words) right here
 - “blkw” means “block of words”

Example Program 1: Summing the Entries in an Array

Let's write an assembly program that computes $0 + 1 + \dots + N$. We will implement the following pseudocode:

Input: N

Output: sum

```
sum = 0
for i = 0 to N (inclusive):
    sum += i
return sum

; Compute 0+1+2+...+N

; R0 holds i
; R1 holds sum

.orig x3000
and r1, r1, 0 ; set sum = 0
and r0, r0, 0 ; set i = 0

add r1, r1, r0 ; sum = sum + i
add r0, r0, 1  ; i = i + 1
; compute i-N, put result in r2
ld r2, 7
not r2, r2
add r2, r2, 1  ; r2 = -N
add r2, r0, r2 ; set cc for i-N
brnz -7        ; repeat loop body if i-N <= 0 aka i <= N

st r1, 1        ; store output (sum) in memory
halt

.blkw 1 ; hold the sum (output)
.fill 3 ; holds input N
.end
```

To determine the PC offsets on the ld, br, st instructions above, we had to count memory locations by hand, and wow, that is annoying! This is why assembly language provides labels. **Labels are names for addresses**. With labels, we could instead write:

```
; Compute 0+1+2+...+N

; R0 holds i
; R1 holds sum

.orig x3000
```

```

and r1, r1, 0 ; set sum = 0
and r0, r0, 0 ; set i = 0

LOOP
add r1, r1, r0 ; sum = sum + i
add r0, r0, 1 ; i = i + 1
; compute i-N, put result in r2
ld r2, N
not r2, r2
add r2, r2, 1 ; r2 = -N
add r2, r0, r2 ; set cc for i-N
brnz LOOP ; repeat loop body if i-N <= 0 aka i <= N

st r1, SUM ; store output (sum) in memory
halt

SUM .blkw 1 ; hold the sum (output)
N .fill 3 ; holds input N
.end

```

This strategy of passing inputs and outputs via ordinary memory locations is unwieldy. Ideally, we would want our assembly programs to read input from the keyboard and print output to the screen. We will tackle this in the next lecture.